HID_keyboard Example Application Note

Revision History

Date	Version	Description
June 30, 2021	0.50	Initial version created

CONFIDENTIAL

©2021, Atmosic Technologies Inc. All rights reserved. Atmosic logo is a registered trademark of Atmosic Technologies Inc. All other trademarks are the properties of their respective holders. This preliminary document is subject to change without notice.

Table of Contents

Overview 4

Quick Start 4

Hierarchy and Files 6

Parameters configurations 8

State Machines 8

kbd_mmi State Machine 8

kbd_gap State Machine 11

Power management 13

Configure Flash Layout 13

Implementation Notes 14

Bluetooth Initialization 14

GAP Parameters 17

Advertisements 17

Connection Parameter Negotiation 17

Device Information Service 18

Key handling 18

CONFIDENTIAL

List of Figures

Figure 1 - Booting
Figure 2 - Connect and Pair Successfully
Figure 3 - Idle Timeout Disconnect
Figure 4 - Keyboard Example Hierarchy
Figure 5 - kbd_mmi State Transitions (Ordinary Cases)
Figure 6 - kbd_mmi State Transitions (Unordinary Cases)
Figure 7 - kbd_mmi Transition Table
Figure 8 - kbd_gap State Transition
Figure 9 - Bluetooth Initialization
Figure 10 - Second Argument to atm_gap_start()
Figure 11 - atm_adv Callback Function
Figure 12 - Connection Parameter
Figure 13 - Device Information Service
Figure 14 - Key Scan Matrix Software Hierarchy

List of Tables

Table 1 - Modules' Functionality
Table 2 - kbd_mmi States
Table 3 - kbd_mmi Operations
Table 4 - kbd_gap States
Table 5 - kbd_gap Operations
Table 6 - Power States
Table 7 - Callback Function to atm_gap.
Tabel 8 - atm_adv Functions
Table 9 - Virtual Key Tables
Table 10 - kbd_unconn_vkey Virtual Key Table
Table 11 - kbd_hid_ready_vkeys Virtual Key Table
Table 12 - kbd_pairing_vkeys Virtual Key Table
Table 13 - kbd_rftest_vkeys Virtual Key Table
Table 14 - kbd_passkey_vkeys Virtual Key Table

CONFIDENTIAL

Overview

This application note describes the settings, functionality, and code flow of the HID_keyboard example code running on an Atmosic ATM2/M3. The Atmosic HID_keyboard example is developed as a Bluetooth keyboard which is compatible with the host supporting BLE HOGP profile.

Quick Start

- Install Atmosic SDK4.1.0
- Go to the HID_keyboard folder of Atmosic SDK and type "**make clean**" then "**make run_all**" to program flash (see Configure Flash Layout). After resetting, it will automatically start sending pairing advertisements with "Atmosic KBD 101" as the device name.
- Console message
    - Booting, see Figure 1.

CONFIDENTIAL

```
@0000002d ATM3221-x1x silicon: 6x6 pkg, External flash
@000000d6 SDK Version: 4.1.0
@0000012a APP Version: 0.0.0.9
6x6 EXT_FLASH: 4e 56 44 53 02 06 0f 41 74 6d 6f 73 69 63 20 4b ...
@0000027a Keyboard HW: 9 ROWS, 17 COLS
@000002ee Keyboard SW: 8 ROWS, 18 COLS
@0000039c [   kbd_mmi][D]: MMI_S_BOOTED (MMI_OP_INITING)
@00000479 [   kbd_mmi][D]: MMI_S_INITING (MMI_OP_INITING)
@000005c0 [ble_gap_se][N]: BOND MASK : 0
@0001c85f [   atm_gap][W]: Unhandled GAPM msg 0xd2c
@0001cf2b [   atm_gap][W]: Unhandled GAPM msg 0xd1c
@0001d004 [   atm_gap][W]: Unhandled GAPM msg 0xd1c
@0001d0c5 [  kbd_hogp][D]: BLE_HOGPD_IDLE
@0001d1dd [   atm_gap][W]: Unhandled GAPM msg 0xd1c
@0001d2a1 +otaps init: hdl:0, app_task:4, sec:4
@0001d36f dts: +dts, maxc:5 attc:7 atth:56
@0001d3f9 dts: found port type:1,1:512, att:2
@0001d48c dts: found port type:3,1:512, att:5
@0001d51d otaps: bulk hdl:2 mbox hdl:5
@0001d597 UPGD init mx_wr:2048, mx_len:512
@0001d626 UPGD init a_off: 0x0, i_off:0x40000, p_size:0x40000, e_size: 4096
@0001d714 UPGD init nv_off: 0x78000, size:32768
@0001d7ae UPGD init debug: 0x800000f
@0001d826 [   atm_gap][W]: Unhandled GAPM msg 0xd1c
@0001d8e4 [   kbd_gap][N]: GAP_S_IDLE (GAP_OP_INITED)
@0001d9c6 lock slots of (hibernation, retention, sleep): 0, 0, 0
@0001da8e [   kbd_gap][D]: [1] Pairing
ext_flash_cmp_write(@0x78036, 2): 00 00
@0001dbb5 [   kbd_mmi][D]: MMI_S_IDLE (MMI_OP_INIT_DONE)
@0001dc9f [   kbd_gap][N]: FW: 0.1.8.7
@0001dda8 [   atm_adv][D]: Adv0: ON (0)
@0001de46 [   kbd_gap][N]: GAP_S_ADV1ING (GAP_OP_ADV1ING)
@0001df33 lock slots of (hibernation, retention, sleep): 0x2, 0, 0
@0001e001 [   kbd_mmi][D]: MMI_S_PAIRING (MMI_OP_PAIRING)
```

Figure 1 - Booting

- Connect and Pair successfully, see Figure 2.

```
@00655868 [   atm_gap][D]: + Peer (0) 7C:B2:7D:46:61:FC
@00655939 [   atm_gap][D]: + Connection interval +    48 (unit:1.25ms)
@00655a2d [   atm_gap][D]: + Slave latency       +     0
@00655af5 [   atm_gap][D]: + Supervision timeout +   960 (unit:  10ms)
@00655bee [   kbd_gap][N]: GAP_S_CONNECTED (GAP_OP_CONNECTED)
@00655ce6 lock slots of (hibernation, retention, sleep): 0x2, 0, 0
@00655db8 [   kbd_mmi][D]: MMI_S_PAIRING (MMI_OP_CONNECTED)
@00655eaa [   atm_gap][D]: ConnInd idx: 0 role: S
@00655f64 [   atm_adv][D]: Adv0: OFF (0)
@00656003 [   kbd_gap][N]: GAP_S_CONNECTED (GAP_OP_ADV_STOP)
@006560f8 lock slots of (hibernation, retention, sleep): 0x2, 0, 0
@006561dc ble_get_handler - Unhandled msgid 0xe01 from 0xff
@006562e7 [  app_bass][V]: app_bass_send_lvl_cb: result 3.352V, Capacity 100.0% 100%
@0065643e otaps: idx:0 ENABLE: 0
@006564b5 [  kbd_hogp][D]: BLE_HOGPD_ENABLED
@0065657e [   atm_gap][V]: atm_gapc_cmp_evt_lower_sniff_latency_locally: status (0)
@00656a7c [   atm_gap][W]: Unhandled GAPC msg 0xe2c
@00656b51 [   atm_gap][W]: Unhandled GAPC msg 0xe2c
@00658931 ble_msg_handler - Unhandled msgid 0xc02 from 0x7
@006590ff [   kbd_gap][N]: New pairing
@006677d0 [   atm_gap][D]: + Peer (0) 7C:B2:7D:46:61:FC
@006678a1 [   atm_gap][D]: + Connection interval +     6 (unit:1.25ms)
@00667993 [   atm_gap][D]: + Slave latency       +    66
@00667a5d [   atm_gap][D]: + Supervision timeout +   300 (unit:  10ms)
@0068e563 [   kbd_mmi][D]: MMI_S_CONNECTED (MMI_OP_PAIR_SUCCESS)
```

Figure 2 - Connect and Pair Successfully

- Idle timeout disconnect, see Figure 3.

```
@01967edf [   kbd_mmi][N]: idle too long timeout

@01967f97 [   kbd_mmi][D]: MMI_S_DISCONNING (MMI_OP_DISCONNING)
@01968177 [   kbd_gap][N]: GAP_S_IDLE (GAP_OP_DISCONNECTED)
@0196826a lock slots of (hibernation, retention, sleep): 0, 0, 0
@01968334 [   kbd_mmi][D]: MMI_S_IDLE (MMI_OP_DISCONNED)
@0196841f [   atm_gap][D]: Disconnect Ind. (0X16)
@019684d7 [   atm_gap][V]: atm_gapc_cmp_evt_disconnect: status (0)
@019685c2 ble_get_handler - Unhandled msgid 0xe03 from 0xff
@0196867c [  kbd_hogp][D]: BLE_HOGPD_IDLE
@0196873c [   kbd_mmi][V]: kbd_mmi_enter_hib
```

Figure 3 - Idle Timeout Disconnect

Hierarchy and Files

The keyboard example is designed for ease of development of customer final products. Every module is designed to be a single functionality toward modules in the driver and atm_ble layer. Figure 4 shows the hierarchy of modules in remote example.

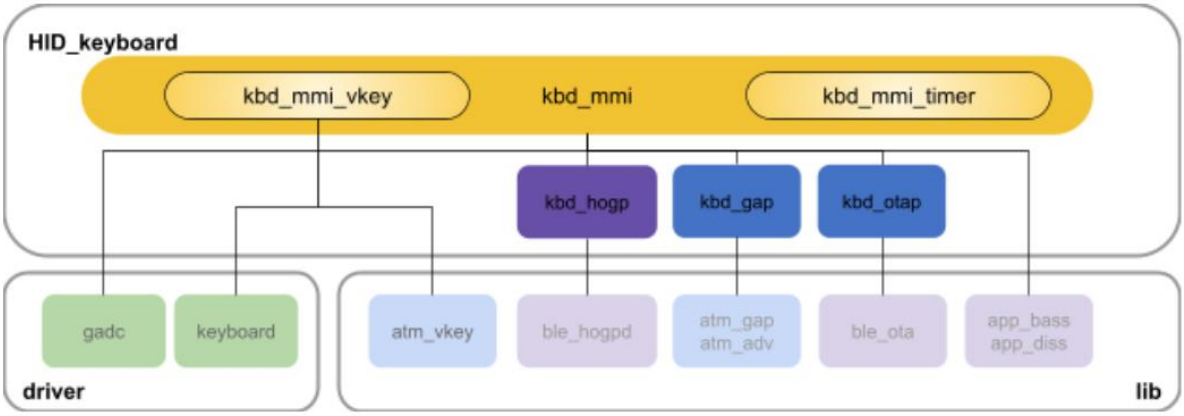


Figure 4 - Keyboard Example Hierarchy

Table 1 describes the module's functionality.

Directory	File Name	Description
src/	kbd_mmi.c kbd_mmi.h	Interacting with all other modules to maintain the flow according to MMI states.
	kbd_mmi_vkey.c kbd_mmi_vkey.h kbd_mmi_vkey_4_4.h kbd_mmi_vkey_8_18.h kbd_mmi_vkey_13_10.h	Handle virtual key behavior in different MMI states. Key code definitions of different products.
	kbd_mmi_timer.c kbd_mmi_timer.h	Handle MMI timer of Atmosic keyboard reference design.
	app_config.h	Bluetooth parameters.
src/bt/	kbd_gap.c kbd_gap.h	Agent of GAP. It provides related operations for kbd_mmi use and maintains Bluetooth states and informs kbd_mmi while changed.
	kbd_hogp.c kbd_hogp.h	Agent of HOGP. It provides HID functionality to kbd_mmi and kbd_mmi_vkey to send reports to the host.
	kbd_otaps.c kbd_otaps.h	Specific OTA parameters to ble_ota.

Table 1 - Modules' Functionality

CONFIDENTIAL

Parameters configurations

In this application, variable configurations are supported and these configurations are defined in `app_config.h`

State Machines

The state machines are the core of application. They represent the reflection of the production situation at some time point. After well understanding state machines, users could easily change and optimize design. State machines of kbd_mmi and kbd_gap are created by application through defining state transition table mechanisms provided by the atm_asm module. Next, two state machines which are built-in the corresponding modules are described.

kbd_mmi State Machine

There are 9 states and 15 operations defined in kbd_mmi state machines. Table 2 describes those states.

CONFIDENTIAL

States	Description
MMI_S_BOOTED	This is the initial state, the chip booted from hardware reset, power on reset or hibernate.
MMI_S_INITING	Initialize kbd_ksm, kbd_gap and other modules.
MMI_S_IDLE	Initialization done or enter hibernate. It also acts as an end state.
MMI_S_PAIRING	Bonding with new host.
MMI_S_RECONNING	Reconnecting the bonded host.
MMI_S_CONNECTED	BT linked is connected but HOGP is not ready.
MMI_S_HID_READY	HOGP is ready.
MMI_S_DISCONNING	Keyboard is initiating the disconnect request
MMI_S_RF_TEST	Enter RF test mode

Table 2 - kbd_mmi States

Operations are used to make the state machine transited when something happens. Each operation was triggered by some modules with a certain meaning. Table 3 describes the operations and the triggered module.

Operations	Description	Trigger by
MMI_OP_INITING	Initialization is ongoing.	kbd_mmi
MMI_OP_INIT_DONE	Initialization is complete.	kbd_gap
MMI_OP_RECONNING	Device is under advertising for reconnection.	kbd_gap
MMI_OP_PAIRING	Device is under advertising for pairing.	kbd_gap
MMI_OP_RECONN_FAIL	Device is fail to reconnect	kbd_gap
MMI_OP_PAIR_FAIL	Device is fail to pair	kbd_gap
MMI_OP_PAIR_ENTER_KEY_START	Device starts to enter pair key	kbd_gap
MMI_OP_PAIR_ENTER_KEY_DONE	Device end to enter pair key	kbd_gap
MMI_OP_PAIR_SUCCESS	Device is paired successfully.	kbd_gap
MMI_OP_CONNECTED	Device is connected.	kbd_gap
MMI_OP_DISCONNED	Link is disconnected	kbd_gap
MMI_OP_HID_READY	HID is ready to use.	KBD_HOGP
MMI_OP_HID_UNREADY	HID becomes unready.	KBD_HOGP
MMI_OP_DISCONNING	MMI wants to disconnect the link. It is caused by an expired IDLE timer.	kbd_mmi
MMI_OP_ADV_STOPPED	Advertisement is stopped normally.	kbd_gap

Table 3 - kbd_mmi Operations

Triggering those operations will cause state transitions. Figure 5 shows the ordinary transition paths of kbd_mmi states. Figure 6 shows the transition paths of kbd_mmi states in unordinary error cases.

CONFIDENTIAL

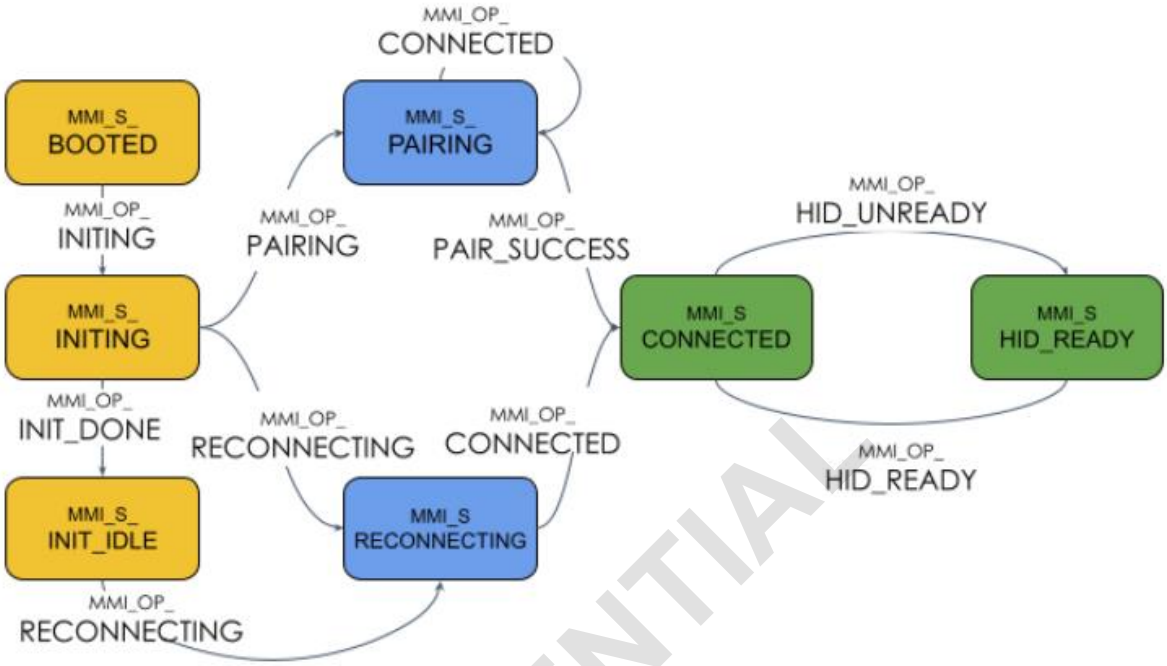


Figure 5 - kbd_mmi State Transitions (Ordinary Cases)

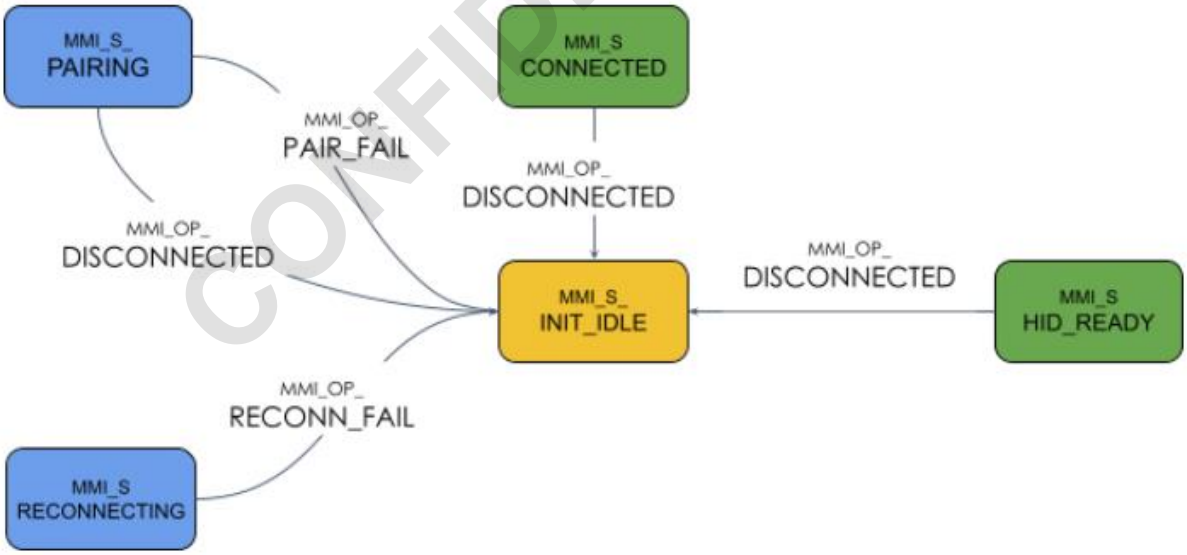


Figure 6 - kbd_mmi State Transitions (Unordinary Cases)

Figure 7 shows the kbd_mmi state transition table mmi_s_tbl[]. There are many expected states and transitions defined in the state transition table. The state transition will happen when specific operation is received. For example, when receiving MMI_OP_PAIR_SUCCESS operation in the MMI_S_PAIRING state, the state machine will move to MMI_S_CONNECTED state. Refer to the source code for more information.

CONFIDENTIAL

```
state_entry const mmi_s_tbl[] = {
    {S_OP(MMI_S_BOOTED, MMI_OP_INITING), MMI_S_INITING, mmi_s_booted_op_initing},
    {S_OP(MMI_S_IDLE, MMI_OP_RECONNING), MMI_S_RECONNING, NULL},
    {S_OP(MMI_S_IDLE, MMI_OP_PAIRING), MMI_S_PAIRING, NULL},
    {S_OP(MMI_S_INITING, MMI_OP_INIT_DONE), MMI_S_IDLE, NULL,},
    {S_OP(MMI_S_INITING, MMI_OP_RECONNING), MMI_S_RECONNING, NULL},
    {S_OP(MMI_S_INITING, MMI_OP_PAIRING), MMI_S_PAIRING, NULL},
    {S_OP(MMI_S_RECONNING, MMI_OP_CONNECTED), MMI_S_CONNECTED, mmi_s_connected},
    {S_OP(MMI_S_RECONNING, MMI_OP_RECONN_FAIL), MMI_S_IDLE, NULL},
    {S_OP(MMI_S_RECONNING, MMI_OP_ADV_STOPPED), MMI_S_IDLE, NULL},
    {S_OP(MMI_S_PAIRING, MMI_OP_CONNECTED), MMI_S_PAIRING, NULL},
    {S_OP(MMI_S_PAIRING, MMI_OP_DISCONNED), MMI_S_IDLE, mmi_s_disconnected},
#ifdef CFG_HID_PASSKEY
    {S_OP(MMI_S_PAIRING, MMI_OP_PAIR_ENTER_KEY_START), MMI_S_PAIRING_PASSKEY, NULL},
    {S_OP(MMI_S_PAIRING_PASSKEY, MMI_OP_PAIR_SUCCESS), MMI_S_CONNECTED, NULL},
    {S_OP(MMI_S_PAIRING_PASSKEY, MMI_OP_DISCONNED), MMI_S_IDLE, mmi_s_disconnected},
    {S_OP(MMI_S_PAIRING_PASSKEY, MMI_OP_PAIR_FAIL), MMI_S_IDLE, NULL},
    // This is repaired.
    {S_OP(MMI_S_CONNECTED, MMI_OP_PAIR_ENTER_KEY_START), MMI_S_PAIRING_PASSKEY, NULL},
#endif
    {S_OP(MMI_S_PAIRING, MMI_OP_PAIR_FAIL), MMI_S_IDLE, NULL},
    // These two states happens after pairing fail.
    {S_OP(MMI_S_CONNECTED, MMI_OP_PAIR_FAIL), MMI_S_IDLE, NULL},
    {S_OP(MMI_S_IDLE, MMI_OP_DISCONNED), MMI_S_IDLE, mmi_s_disconnected},
    // This is repaired.
    {S_OP(MMI_S_CONNECTED, MMI_OP_PAIR_SUCCESS), MMI_S_CONNECTED, NULL},
    {S_OP(MMI_S_PAIRING, MMI_OP_PAIR_SUCCESS), MMI_S_CONNECTED, NULL},
    {S_OP(MMI_S_PAIRING, MMI_OP_ADV_STOPPED), MMI_S_IDLE, NULL},
    {S_OP(MMI_S_CONNECTED, MMI_OP_HID_READY), MMI_S_HID_READY, mmi_s_hid_ready},
    {S_OP(MMI_S_CONNECTED, MMI_OP_DISCONNED), MMI_S_IDLE, mmi_s_disc_restart},
    {S_OP(MMI_S_CONNECTED, MMI_OP_DISCONNING), MMI_S_DISCONNING, mmi_s_discing},
    {S_OP(MMI_S_HID_READY, MMI_OP_DISCONNING), MMI_S_DISCONNING, mmi_s_discing},
    {S_OP(MMI_S_HID_READY, MMI_OP_DISCONNED), MMI_S_IDLE, mmi_s_disc_restart},
    {S_OP(MMI_S_DISCONNING, MMI_OP_DISCONNED), MMI_S_IDLE, mmi_s_disconnected},
    {S_OP(MMI_S_RF_TEST, MMI_OP_DISCONNED), MMI_S_RF_TEST, NULL},
};
```

Figure 7 - kbd_mmi Transition Table

kbd_gap State Machine

Like kbd_mmi, the state machine of kbd_gap is created in the application layer. There are seven states and 9 operations defined in kbd_gap state machines. Table 4 describes those states.

States	Description
GAP_S_INIT	Initial state.
GAP_S_IDLE	No Bluetooth activity is ongoing. Go to hibernate if not further request from kbd_mmi.
GAP_S_ADV0ING	Pairing advertisements is ongoing.
GAP_S_ADV1ING	Reconnecting advertisements is ongoing.
GAP_S_ADV_STOPPING	Advertisement is stopping actively.

GAP_S_CONNECTED	ACL link exists.
GAP_S_RFTEST	DTM mode is ongoing.

Table 4 - kbd_gap States

Each operation was triggered by some atm_gap callbacks with a certain meaning. Table 5 describes the operations and the triggering function.

Operations	Description	Trigger by
GAP_OP_INITING	atm_gap is initialing	kbd_gap_init
GAP_OP_INITED	atm_gap initialization indication	kbd_gap_init_cfm
GAP_OP_ADV0ING	Pairing started confirmation	adv_cfm
GAP_OP_ADV1ING	Reconnecting started confirmation	adv_cfm
GAP_OP_ADV_STOP	ADV stopped by connection	adv_cfm
GAP_OP_ADV_STOP_TOUT	ADV stopped by timeout	adv_cfm
GAP_OP_ADV_STOPPING	ADV Stopping is ongoing by kbd_gap	kbd_gap_stop_discoverable
GAP_OP_CONNECTED	ACL link connection indication	kbd_conn_ind
GAP_OP_DISCONNED	ACL link disconnection indication	kbd_disc_ind

Table 5 - kbd_gap Operations

Figure 8 shows the transition paths of kbd_gap states.

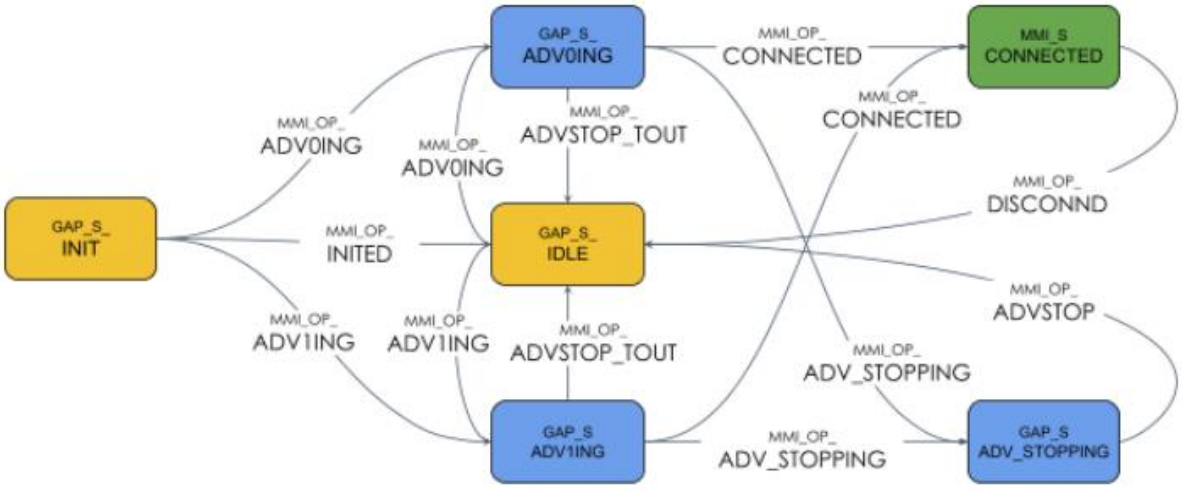


Figure 8 - kbd_gap State Transition

CONFIDENTIAL

Power management

In this example code, the atm_pm module's lock scheme is used to manage whether or not to prevent entering a power state. Three kinds of locks from high to low are PM_LOCK_SLEEP, PM_LOCK_RETENTION and PM_LOCK_HIBERNATION which are associated with sleep, retention and hibernation. Once all the locks in one level are unlocked and no higher level lock is locked, the system will go into that power state. Table 6 lists all the locks used in this example and descriptions:

PM_LOCK_SLEEP		
Name	Module	Description
ksm_lock_sleep	driver/keyboard	Mostly stay unlocked. Only be locked for key interrupt processing.
PM_LOCK_HIBERNATION		
Name	Module	Description
vk_lock_hiber	driver/atm_vkey	Locked when virtual keys are not all released.
kbd_gap_lock_hib	kbd_gap	Locked when kbd_gap state is not GAP_S_IDLE.

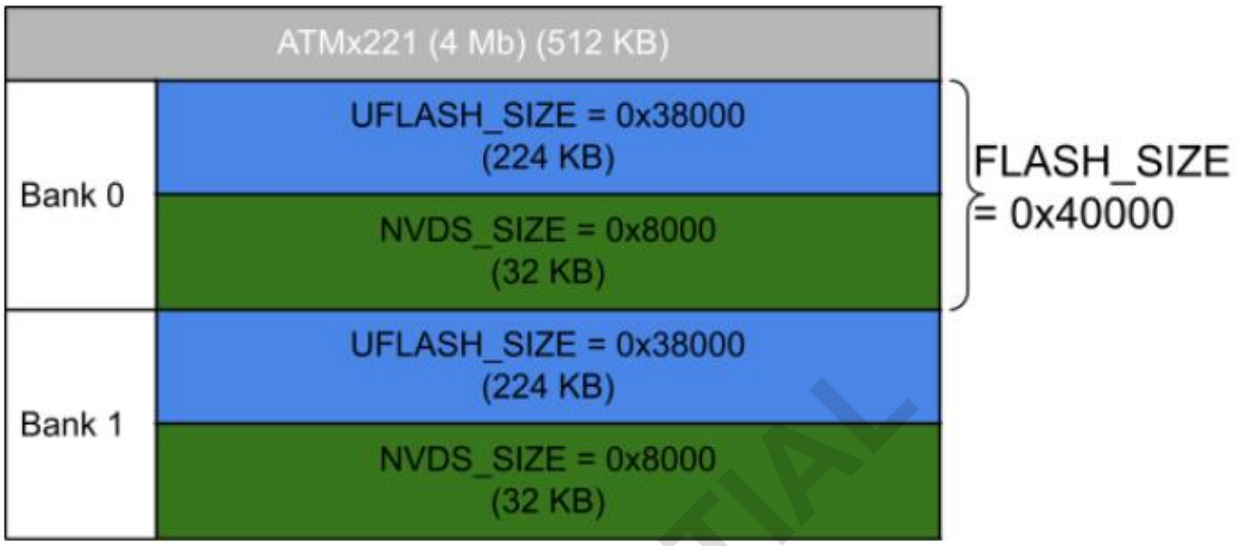
Table 6 - Power States

CONFIDENTIAL

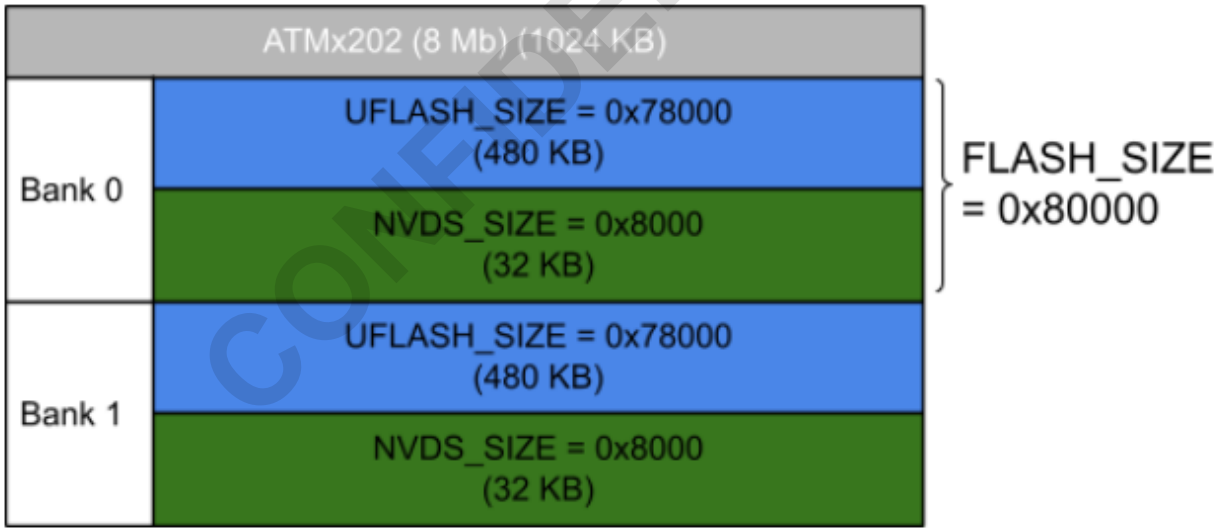
Configure Flash Layout

The OTA feature is enabled by default. The flash essentially utilizes half of its size. The makefile uses 256 KB (FLASH_SIZE=0x40000) and 512 KB (FLASH_SIZE=0x80000) for ATMx221 and ATMx202 respectively by default. The default NVDS_SIZE is 0x8000.

Default layout of ATMx221:



Default layout of ATMx202:



CONFIDENTIAL

Implementation Notes

Bluetooth Initialization

The Bluetooth initialization is started in `kbd_gap_init()` which is called when entering MMI_S_INITING state. In `kbd_gap_init()`, it registers profiles by calling `atm_gap_prf_reg()`, then calls `atm_gap_start()` to start the Bluetooth initialization procedure, as shown in Figure 9.

```
void kbd_gap_init(void)
{
    /*Reg. profiles */
    atm_gap_prf_reg(BLE_DISS, kbd_dis_param());
    atm_gap_prf_reg(BLE_BASS, kbd_bass_param());
    atm_gap_prf_reg(BLE_HOGPD, kbd_hogp_param());
    atm_gap_prf_reg(BLE_OTAPS, kbd_otps_param());

    /*Start */
    atm_gap_start(atm_gap_param_get(), &gap_callbacks);

    /*KBD_GAP state machine init*/
    atm_asm_init_table(GAP_S_TBL_IDX, gap_s_tbl,
            sizeof(gap_s_tbl) / sizeof(gap_s_tbl[0]));
    atm_asm_set_current(GAP_S_TBL_IDX, GAP_S_INIT, GAP_OP_INITING);
    atm_asm_reg_state_change_cb(GAP_S_TBL_IDX, kbd_gap_s_changed);
    kbd_gap_lock_hiber = atm_pm_alloc(pm_lock_hibernate);
    atm_pm_lock(kbd_gap_lock_hiber);
}
```

Figure 9 - Bluetooth Initialization

Two arguments are passed into `atm_gap_start()`. First is the type of `atm_gap_param_t` and it is the configuration parameters used to configure the device. Second is the callback function that atm_gap will invoke on Bluetooth status changes. As to the second argument, kbd_gap provides a set of callbacks to atm_gap as shown on Figure 10, and described in Table 7.

```
static atm_gap_cbs_t const gap_callbacks = {
    .conn_ind = kbd_conn_ind,
    .disc_ind = kbd_disc_ind,
    .pair_req_ind = kbd_pair_req_ind,
#ifdef CFG_HID_PASSKEY
    .pair_passkey_ind = kbd_pair_passkey_ind,
#endif
    .pair_numeric_ind = kbd_pair_numeric_ind,
    .pair_ind = kbd_pair_ind,
    .conn_param_updated_ind = kbd_gap_conn_param_updated_ind,
    .init_cfm = kbd_gap_init_cfm,
};
```

Figure 10 - Second Argument to `atm_gap_start()`

Name	Description
_kbd_gap_init_cfm	Initialization confirmation.
_kbd_conn_ind	Connection indication.

_kbd_disc_ind	Disconnection indication.
_kbd_pair_req_ind	Pairing request indication.
_kbd_gap_conn_param_updated_ind	Connection parameter updated indication.
_kbd_pair_ind	Paring result indication.
_kbd_pair_passkey_ind	Passkey display or input. Due to the board level limitation.. This is not used for the current setting. If there are ways to display or input numbers, users could change the content of _kbd_pair_req_ind from “no input/output” to display type IO capability.
_kbd_pair_numeric_ind	Numeric comparison indication. Call ble_gap_sec_number_rsp(..) to confirm it is valid. This is also same as _kbd_pair_passkey_ind

Table 7 - Callback Function to atm_gap

CONFIDENTIAL

GAP Parameters

Advertisements

The kbd_gap uses the atm_adv module to achieve discoverability for the host. kbd_gap Implements the advertising callback functions that catch the states, and register it to the system through atm_adv_reg() function. Figure 11 shows the callback function. atm_adv provides a set of easy used functions, users could find them from source files. See Table 8 for atm_adv functions descriptions.

```
/// Advertising callback function
typedef void (*atm_adv_cb_t)(atm_adv_state_t state, uint8_t act_idx,
    enum hl_err status);
```

Figure 11 - atm_adv Callback Function

API Name	Description
uint8_t atm_adv_reg(atm_adv_reg_t*);	Register advertising callback function
uint8_t atm_adv_create(atm_adv_create_t*);	Create advertising activity
uint8_t atm_adv_set_adv_data(uint8_t act_idx, atm_adv_data_t* p)	Set the advertising data
uint8_t atm_adv_set_scan_data(uint8_t act_idx, atm_adv_data_t* p)	Set the scan response data
uint8_t atm_adv_start(uint8_t act_idx, atm_adv_start_t* p)	Start advertising
uint8_t atm_adv_stop(uint8_t act_idx)	Stop advertising
uint8_t atm_adv_delete(uint8_t act_idx)	Delete advertising

Tabel 8 - atm_adv Functions

Connection Parameter Negotiation

Four parameters are related to the connection parameters negotiation: CFG_GAP_CONN_INT_MIN, CFG_GAP_CONN_INT_MAX, CFG_GAP_CONN_TIMEOUT, and CFG_GAP_SLAVE_LATENCY. After connecting with Bluetooth LE master, the device will perform connection parameter update negotiation. In `kbd_gap.c` - `kbd_gap_connect_param_nego()`, param_nego is the parameter for connection parameter negotiation. Customers can modify the parameter depending on the application. See Figure 12.

```
static atm_gap_param_nego_t const param_nego = {
```

CONFIDENTIAL

```
    .param_nego_cfm = kbd_gap_param_nego_cfm,
    .retry_times = KBD_PARAM_NEGO_TIMES,
    .check_result = KBD_PARAM_NEGO_TOUT_CS,
    .target = &(struct gapc_conn_param) {
        .intv_min = CFG_GAP_CONN_INT_MIN,
        .intv_max = CFG_GAP_CONN_INT_MAX,
        .latency = CFG_GAP_SLAVE_LATENCY,
        .time_out = CFG_GAP_CONN_TIMEOUT,
    }
};
```

Figure 12 - Connection Parameter

Device Information Service

The strings of device information service such as manufacture name, model name, firmware revision, software revision, etc., are defined with APP_DIS_*, and can be modified by users if needed. See Figure 13.

```
#define APP_DIS_MANUFACTURER_NAME "Atmosic Tech."
#define APP_DIS_MODEL_NB_STR "KBD-01"
#define APP_DIS_SERIAL_NB_STR "1.0.0.0"
#define APP_DIS_FIRM_REV_STR "0.1.8.7"
#define APP_DIS_SW_REV_STR "0.1.8.7"
#define APP_DIS_HARD_REV_STR "1.0.0"
#define APP_DIS_SYSTEM_ID "\x12\x34\x56\xFF\xFE\x9A\xBC\xDE"
#define APP_DIS_IEEE "\xFF\xEE\xDD\xCC\xBB\xAA"
```

Figure 13 - Device Information Service

CONFIDENTIAL

Key handling

The keyboard example uses three modules to complete the user input via key scan matrix: ksm, atm_vkey and kbd_mmi_vkey. The ksm is a key scan matrix hardware driver. And the atm_vkey is a framework module which handles key behavior such as click, double click, hold...etc, and generates virtual key events to the upper layer. And the kbd_mmi_vkey is the application module which is designed to listen to key matrix events then pass it to atm_vkey. Figure 14 shows the hierarchy.

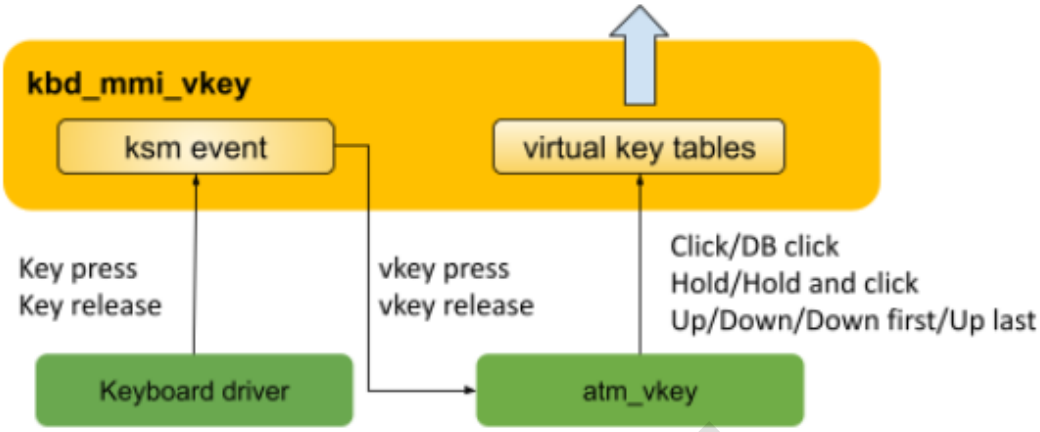


Figure 14 - Key Scan Matrix Software Hierarchy

The atm_vkey handles the key behavior and generates events according to the virtual key table which was registered by kbd_mmi_vkey. In initialization, kbd_mmi_vkey registers three virtual key tables into atm_vkey.

For kbd_mmi_vkey, these virtual key tables represent the different MMI states.
In different MMI states, the kbd_mmi_vkey will pass vkey into atm_vkey with a specified virtual key table. Table 9 lists five virtual key tables in different MMI states. Table 10, 11, 12, 13 and 14 list registered keys in each virtual key table.

Virtual Key Table	MMI states
kbd_unconn_vkey	MMI_S_BOOTED MMI_S_INITING MMI_S_IDLE MMI_S_RECONNING MMI_S_CONNECTED MMI_S_DISCONNING
kbd_hid_ready_vkeys	MMI_S_HID_READY
kbd_pairing_vkeys	MMI_S_PAIRING
kbd_rftest_vkeys	MMI_S_RF_TEST
kbd_passkey_vkeys	MMI_S_PAIRING_PASSKEY

Table 9 - Virtual Key Tables

kbd_unconn_vkey

CONFIDENTIAL

Key Behavior	Action
DOWN_FIRST_SEND	Save the key.
DOWN_MORE_SEND	Remove all bonded information
UP_INTER_SEND	Save the key.
UP_LAST_SEND	Save the key.

Table 10 - kbd_unconn_vkey Virtual Key Table

kbd_hid_ready_vkeys	
Key Behavior	Action
DOWN_FIRST_SEND	Send report with corresponding code
DOWN_MORE_SEND	Send report with corresponding code
UP_INTER_SEND	Send report with 0 (key up)
UP_LAST_SEND	Send report with 0 (key up)
HOLD_CLICK_BOND_FN1	Bonding control, please refer to kbd_function_handle
HOLD_CLICK_RF_TEST	Enter RF test mode, please refer to kbd_enter_rftest
HOLD_CLICK_BOND_FN2	Bonding control, please refer to kbd_function_handle

Table 11 - kbd_hid_ready_vkeys Virtual Key Table

CONFIDENTIAL

kbd_pairing_vkeys	
Key Behavior	Action
HOLD_CLICK_BOND_FN1	Bonding control, please refer to kbd_function_handle
HOLD_CLICK_RF_TEST	Enter RF test mode, please refer to kbd_enter_rftest
HOLD_CLICK_BOND_FN2	Bonding control, please refer to kbd_function_handle

Table 12 - kbd_pairing_vkeys Virtual Key Table

kbd_rftest_vkeys	
Key Behavior	Action
UP_LAST_SEND	RF test control, please refer to kbd_rftest

Table 13 - kbd_rftest_vkeys Virtual Key Table

kbd_passkey_vkeys	
Key Behavior	Action
DOWN_FIRST_SEND	Used to enter passkey, please refer to kbd_pass_key

Table 14 - kbd_passkey_vkeys Virtual Key Table

CONFIDENTIAL



CONFIDENTIAL

ATMOSIC TECHNOLOGIES - DISCLAIMER

This product document is intended to be a general informational aid and not a substitute for any literature or labeling accompanying your purchase of the Atmosic product. Atmosic reserves the right to amend its product literature at any time without notice and for any reason, including to improve product design or function. While Atmosic strives to make its documents accurate and current, Atmosic makes no warranty or representation that the information contained in this document is completely accurate, and Atmosic hereby disclaims (i) any and all liability for any errors or inaccuracies contained in any document or in any other product literature and any damages or lost profits resulting therefrom; (ii) any and all liability and responsibility for any action you take or fail to take based on the information contained in this document; and (iii) any and all implied warranties which may attach to this document, including warranties of fitness for particular purpose, non-infringement and merchantability. Consequently, you assume all risk in your use of this document, the Atmosic product, and in any action you take or fail to take based upon the information in this document. Any statements in this document in regard to the suitability of an Atmosic product for certain types of applications are based on Atmosic's general knowledge of typical requirements in generic applications and are not binding statements about the suitability of Atmosic products for any particular application. It is your responsibility as the customer to validate that a particular Atmosic product is suitable for use in a particular application. All content in this document is proprietary, copyrighted, and owned or licensed by Atmosic, and any unauthorized use of content or trademarks contained herein is strictly prohibited.

Copyright ©2021 by Atmosic Technologies. All rights reserved.

www.atmosic.com